Software Test Plan (STP) - Online Bookstore

Project: Online Bookstore
Version: 1.0
Authors: Vihaan Wadhwa, Abdullah Faruqui, Nikhil Tej Vaddi, Deepthi A. E.
Date: 01-10-2026
Status: Draft

1. Introduction

Purpose: This document defines the test plan for the Online Bookstore v1.0. It specifies the testing objectives, scope, strategy, environment, schedule, deliverables, responsibilities, risks, traceability, and metrics for verifying the system against the SRS and Software Architecture and Design Specification (SAD).

Scope: Testing covers buyer, seller, and administrator functionality including registration and authentication, catalogue and search, cart and checkout, order management and tracking, seller listing management, administrator approval and inventory management, reviews, wishlist, and notifications. It also covers the specified performance, scalability, availability, accessibility, maintainability, backup/recovery, and security requirements. The internal implementation of third-party payment, email, and shipping services is outside project scope; only the interfaces and failure-handling behavior of the bookstore are tested.

References: Online Bookstore SRS v1.0; Online Bookstore Software Architecture and Design Specification v1.0; PES University Software Engineering mini-project guidelines; project Requirements Traceability Matrix (RTM).

Definitions: SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing), JWT (JSON Web Token), RBAC (Role-Based Access Control), ORM (Object-Relational Mapping), TLS (Transport Layer Security), WCAG (Web Content Accessibility Guidelines), RPO (Recovery Point Objective), RTO (Recovery Time Objective).

2. Test Items

- WebUI / presentation layer
- APIGateway / REST API layer
- AuthModule
- CatalogueModule
- CartModule
- CheckoutModule and PaymentClient
- OrderModule and ShippingClient interface
- SellerModule
- AdminModule
- ReviewModule / Wishlist
- NotificationModule and EmailClient

- Django ORM and PostgreSQL data layer
- Security controls including JWT, RBAC, password hashing, input validation, HTTPS/TLS, and protection against injection/XSS

3. Features to be Tested

Features mapped to SRS requirement IDs:
- BOOK-F-001 to BOOK-F-004: Registration, login, account lockout, password reset
- BOOK-F-005 to BOOK-F-007: Catalogue, search/filtering, book details
- BOOK-F-008 to BOOK-F-010: Cart, checkout, payment and order confirmation
- BOOK-F-011 to BOOK-F-013: Order history, shipment tracking, cancellation/return
- BOOK-F-014 to BOOK-F-017: Seller listings and administrator approval/inventory controls
- BOOK-F-018 to BOOK-F-020: Reviews, wishlist and order/status notifications
- BOOK-NF-001 to BOOK-NF-006: Performance, scalability, availability, accessibility, test coverage, backup/recovery
- BOOK-SR-001 to BOOK-SR-005: TLS, password protection, payment-data protection, RBAC, injection/XSS protection

4. Features Not to be Tested

- Internal implementation of external payment gateway, email/SMTP service, and shipping/logistics provider
- Physical warehouse, courier operations, and third-party service internals
- Real financial transactions; payment testing uses the configured sandbox/test gateway
- Features or future enhancements explicitly listed in the SAD as future ideas, such as two-factor login, wishlist price alerts, and advanced search/caching, unless they are later included in the implemented build

5. Test Approach / Strategy

Levels:
- Unit testing: individual Django functions, classes, modules, validation rules, business-logic functions, and model behavior.
- Integration testing: interactions between WebUI/APIGateway and modules, modules and PostgreSQL through Django ORM, and bookstore adapters such as PaymentClient, EmailClient, and ShippingClient.
- System testing: complete end-to-end buyer, seller, and admin workflows through the deployed application.
- Acceptance testing (UAT): verification of the SRS acceptance criteria and high-priority functional requirements.

Types:
- Functional testing for all implemented functional requirements.
- Regression testing after defect fixes and changes.
- Performance/load testing for response-time and concurrent-user requirements.
- Usability/accessibility testing on desktop and mobile layouts, including WCAG 2.1 AA-related checks.
- Security testing covering authentication, authorization/RBAC, TLS, sensitive-data handling, injection/XSS protection, brute-force controls, and security validation.
- Reliability and recovery testing for availability and backup/RPO/RTO requirements.
- API testing for defined REST endpoints and expected HTTP status/error responses.

Test-case requirement: For each implemented use case/functional requirement, the project will prepare at least 8-10 test cases as required by the PES University Software Engineering mini-project guidelines, including Unit, Integration, and System test cases. Both valid and invalid input conditions will be covered.

Entry Criteria: A build is available and deployable; required database and test environment are available; test data and test accounts are prepared; relevant dependencies/sandboxes are reachable or stubbed; and the feature under test has completed development sufficiently for execution.

Exit Criteria: All planned test cases have been executed; critical/blocking defects are resolved or formally accepted; high-priority functional requirements have been verified; mapped security and NFR tests have been completed; required coverage and performance evidence has been collected; and the RTM has been updated with test results.

5.1 Security Validation

- Verify HTTPS/TLS 1.2+ for client-server and third-party connections.
- Verify passwords are stored using Django's password hashing and are not present in plaintext in logs.
- Verify payment-card data is never stored; only the payment gateway reference is retained.
- Verify server-side RBAC prevents Buyer access to Seller/Admin operations and prevents users from modifying resources they do not own.
- Test input validation and protection against SQL injection and XSS using safe security-test payloads.
- Verify account lockout after five consecutive failed login attempts and the 15-minute lock period.
- Verify sensitive credentials, tokens, gateway keys, and secrets are not written to source control or application logs.
- Perform security validation including appropriate penetration/fuzz testing of authentication, API input, authorization, and checkout interfaces in an isolated test environment.
- Record security findings as defects/issues and retest after fixes.

6. Test Environment

Hardware: Standard development/test laptops or desktops; no specialized hardware is required.
Software: Python, Django, Django REST Framework, Django templates/HTML/CSS, PostgreSQL, JWT authentication, and HTTPS/TLS. The project SAD identifies Render, Heroku, or AWS as possible hosting environments.
Testing tools: Django TestCase / pytest-django with coverage reporting; API testing through the project's REST interfaces; Git/GitHub for version control and defect tracking; Jenkins for CI/CD; Docker for containerization; SonarQube or an equivalent static-analysis tool where configured by the team.
External test services: Razorpay or Stripe sandbox, SMTP/test email service, and shipping service or test stub where applicable.
Test Data: Dedicated Buyer, Seller, and Admin accounts; approved/rejected/pending book listings; stock and out-of-stock books; valid/invalid addresses; sandbox payment tokens; orders in different lifecycle states; review/wishlist data; invalid and security-test input values.

7. Test Schedule

Milestones:
- Test-plan preparation and review: Phase 1

- Test-case design and RTM expansion: Phase 2
- Test-environment and test-data setup: Phase 3
- Unit and integration test execution: Phase 4
- System, performance, usability, and security validation: Phase 5
- Regression testing and defect retesting: Phase 6
- UAT and final test-summary preparation: Phase 7

The exact calendar dates will be aligned with the team's Agile sprint schedule and course submission deadlines. No execution dates are claimed until the team finalizes the sprint/release schedule.

8. Test Deliverables

- Software Test Plan (STP)
- Detailed manual and automated test cases
- Test scripts and automation where applicable
- Controlled test data and test accounts
- Unit/integration test and coverage reports
- API/system/performance/security test evidence
- Test execution logs containing Actual Result and Pass/Fail status
- Defect/bug reports and retest evidence
- Updated Requirements Traceability Matrix
- Final Test Summary Report
- Security validation evidence and findings
- CI/CD and static-analysis evidence where applicable

9. Roles and Responsibilities

Role	Name	Responsibility
Test/QA Lead	Abdullah Faruqui	Maintain the STP, coordinate test planning/execution, and monitor coverage/reporting.
Test Engineer	Deepthi A. E	Design and execute Unit/Integration/System test cases, record results, and report defects.
Developer	Nikhil T. Vaddi	Implement fixes, support debugging, review failures, and perform regression verification.
Project/Requirements Owner	Vihaan Wadhwa	Confirm requirements and acceptance criteria, review test status, and support final sign-off.

10. Risks and Mitigation

Risk	Mitigation
Unstable build or delayed feature delivery	Use incremental Agile builds, smoke tests, and CI checks before detailed test execution.

Test environment/database unavailable	Use Dockerized/reproducible environment, database backup, and a documented recovery procedure.
Payment gateway or external service unavailable	Use sandbox/stubbed services, timeouts, controlled failure responses, and isolated integration tests.
Insufficient or poor-quality test data	Maintain dedicated Buyer/Seller/Admin accounts and controlled datasets for normal, boundary, and invalid cases.
Concurrent checkout causes stock inconsistency	Test transactional stock locking/concurrency behavior and verify rollback/refund handling where applicable.
Security testing exposes sensitive test data	Use isolated test accounts/data; never use real payment-card information or production secrets.

11. Assumptions & Dependencies

- The project will use the Python/Django + PostgreSQL architecture described in the SAD.
- The required test environment and test database will be available before execution.
- Payment testing will use a sandbox/test gateway and no real transactions.
- External services may be unavailable; where necessary, controlled mocks/stubs will be used to test bookstore behavior independently of third-party internals.
- Test accounts and representative book/order data will be created before execution.
- GitHub/Jira or the team's selected issue-tracking workflow will be used to record defects and changes.
- The shipping API implementation remains an open project decision in the SAD; its tests will be limited to the implemented behavior in the final build.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment or database is unavailable and blocks meaningful execution.
- A build failure or blocking defect prevents execution of more than approximately 30% of the planned test cases.
- Required test data or a mandatory dependency is unavailable and cannot be stubbed.
- A security test could affect real users, real payment data, or production resources.

Resume testing if:

- The blocking defect is resolved or an approved workaround is available.
- The environment is stable and the required data/services are restored or stubbed.
- A new build is deployed and smoke testing confirms basic system stability.

13. Test Case Management & Traceability

The RTM will map each SRS requirement to the relevant architecture/design component, code/module where available, and test case IDs. The existing SAD traceability provides the starting mapping. Examples:

- BOOK-F-001 (Register account) → AuthModule → TC-Auth-01 and expanded Unit/Integration/System cases
- BOOK-F-002 (Login/token) → AuthModule/APIGateway → TC-Auth-02 and expanded cases

- BOOK-F-008 (Cart operations) → CartModule → TC-Cart-01 and expanded cases
- BOOK-F-010 (Payment/order confirmation) → CheckoutModule/PaymentClient/OrderModule → TC-Cart-03 and expanded cases
- BOOK-F-016 (Admin approval/rejection) → AdminModule/NotificationModule → TC-Admin-01 and expanded cases
- BOOK-NF-001 (Response time) → WebUI/APIGateway → TC-Perf-01
- BOOK-SR-002 (Password hashing) → AuthModule → TC-Sec-02
- BOOK-SR-004 (RBAC) → AuthModule/APIGateway → TC-Sec-04
- BOOK-SR-005 (Injection/XSS protection) → APIGateway/Django ORM → TC-Sec-05

The final RTM will be updated after implementation and test execution so that every implemented requirement has test coverage and an execution status.

14. Test Metrics & Reporting

Metrics collected:
- Number and percentage of planned test cases executed
- Pass, fail, and blocked percentages
- Requirement coverage and test-case coverage
- Unit/integration/system test distribution
- Code and branch coverage where supported by the configured coverage tool
- Defect count by severity and priority
- Defect density and defect aging
- Regression-test pass rate
- Security-test findings and closure status
- Performance response times, including 90th-percentile targets where applicable
- Concurrent-user/load-test results
- Availability and backup/RPO/RTO evidence

Reports:
- Sprint/test execution status
- Defect reports and retest evidence
- CI/coverage/static-analysis reports
- Security validation report
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
Test/QA Lead	Abdullah F	Pending
Development Lead	Nikhil T. V.	Pending
Project/Requirements Owner	Vihaan W.	Pending